8-Week Journey Summary

TinyML for Real-time Stress Detection on Wearable Devices

Toan Gian

1 Project Overview and Goal

Real-time stress detection on wearable devices can support continuous mental health monitoring and well-being assessment. However, stress inference from wearable physiological signals is constrained by limited compute, memory, and power budgets. Traditional neural networks (NNs) often achieve strong performance but require large storage and computation, which can be infeasible to deploy on microcontrollers commonly used in wearable platforms.
Goal: This project aims to design an effective stress ML-based classifier that runs *always-on* on a resource-constrained wearable device while maintaining reliable classification performance and incurring only a modest increase in computational complexity and energy consumption.
Core Question: How can we build a TinyML-based stress detection model that remains effective while being feasible for on-device deployment under strict resource budgets?
Outcome Summary: This work delivers six main outcomes

- We evaluate several ML approaches, including classical ML models (e.g., Support Vector Machines (SVM) and Random Forests (RF)), a standard CNN baseline, and a lightweight CNN variant based on depthwise separable convolutions (DSConv). We systematically characterize the trade-offs between classification performance and computational cost across both a public dataset and a self-collected dataset. Evaluation uses standard embedded-oriented metrics, including parameter count, MACs, memory footprint, inference latency, and energy-related measurements.

- We investigate model compression strategies, including pruning, quantization, and knowledge distillation, and assess their impact on both accuracy and on-device efficiency.

- We build a self-collected stress versus normal PPG dataset using a MAX30105 sensor and document practical challenges in labeling and deployment.

- We implement a real-time stress classification pipeline on-device using an SVM model, enabling end-to-end operation from data acquisition to on-device decision-making.

- We evaluate the performance and resource consumption of the standard CNN-based model under diverse operating conditions (e.g., batch size, sampling rate, injected noise, and missing-data scenarios).

- We prepared two manuscripts: (i) an analysis comparing DSConv and standard CNN designs, and (ii) an end-to-end, system-level report of the full pipeline and experimental findings.

2 What I Built

2.1 Dataset and pre-processing pipeline.

- **Dataset:**

  1. **WESAD:** It is a published dataset which collected from 15 subjects with **PPG only** to reflect common wearable availability and non-invasive sensing, consisting of 3000 window samples.
  2. **Self-data:** Collected from myself using MAX30105 sensor. This work use a diverse set of short video stimuli designed to elicit different emotional or arousal states (e.g., high-arousal vs. low-arousal). The stimulus condition and timing will be used as a proxy ground truth while physiological signals are recorded, consisting of 800 window samples.

- **Task:** Binary classification (stress vs. no stress), focusing on Baseline (calm) vs. Stress conditions.
- **Sampling rate:** 64 Hz.
- **Segmentation:** Sliding window with window size 512 samples (8 seconds) and 25% overlap (128 samples), step size 384.
- **Normalization:** Standardization using training-set mean and standard deviation to stabilize training and mitigate scale differences.

2.2 Training and evaluation pipeline.

- Implemented in **PyTorch** and trained on GPU for 50 epochs using Adam (batch size 128, learning rate 0.001).
- Reported metrics: Accuracy, Macro-F1, confusion matrix, and model complexity as an efficiency proxy.

2.3 Deployment pipeline (embedded).

- Target device: **Arduino Nano 33 BLE Sense** using **TensorFlow Lite Micro**.
- Prototype components for realistic operation: **MAX30105 PPG sensor** and **OLED display** for live inference feedback, as shown in Fig. 1.
- Model Compression Methods: We implement quantization, pruning, and knowledge distillation to compress model on device. In particular, we use 1) Quantization Aware Training (QAT) relied on its simple implementation; 2) structured pruning because its MCU-friendly, which is prioritized in practical deployment; 3) and knowledge distillation with a 4-CNN blocks teacher model and 2 and 3-CNN blocks students.
- Varied Condition: We evaluate robustness to changes in sampling rate and batch size, including (i) a hop-size sweep and (ii) bursty dataflow variants: (1) bursts at the PC sender (UART bursty arrivals with the same average sampling rate) and (2) bursts at the MCU during inference (batching multiple windows and running back-to-back inferences). We also assess signal robustness under challenging perturbations, including additive noise and missing-data patterns.

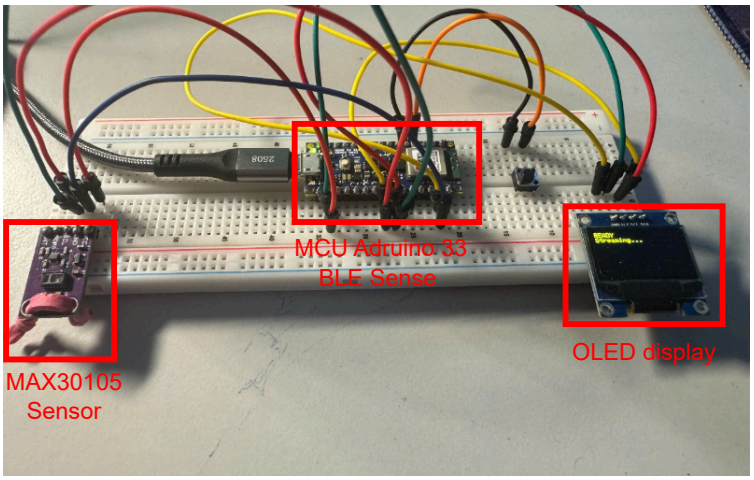

Figure 1: On-devices Setup.

2.4 On-device measurement pipeline.

- **Flash Usage:** program storage.
- **Runtime RAM Usage:** global variables and free dynamic memory.
- **CPU Active Time (%)**: percentage of time the MCU is active (not sleeping) during the measurement window.
- **Inference Time:** measured on device.
- **inf/s:** how many model predictions the Arduino produced per second. (*Note that: Expected value: inf/s = Fs/HOP*)
- **infer_ms/s:** time per second spent inside `interpreter->Invoke()` (arduino loop).
- **rx_ms/s:** milliseconds per second spent reading/parsing incoming serial data.
- **fill_ms/s:** time per second spent preparing the input window (copying N samples, z-score normalization, quantization if int8, etc.).
- **Energy proxy (ms/s)**: aggregated CPU work per second (inference + fill + RX), estimating the compute load in a streaming loop.

3 Machine Learning Models

3.1 Classical ML

For classical ML methods, we use SVM and RF to evaluate performance on WESAD as well as self-collected data. Furthermore, fixed-length feature vectors were extracted from each PPG window. These handcrafted features summarize both the amplitude and temporal characteristics of the signal and are computationally lightweight. The extracted features include:

- **Statistical features:** mean, standard deviation, minimum, maximum, median.
- **Signal dynamics:** first-order difference mean and standard deviation
- **Energy-related features:** signal energy and root mean square (RMS)
- **Shape-related features:** skewness and kurtosis

3.2 DL-based Models

For DL-based models, we use standard CNN and DSConv to evaluate the performance and complexity on WESAD and self-collected data. Notably, DSConv reduces parameter cost from roughly $k \cdot F_{in} \cdot F_{out}$ (i.e., standard CNN) to $k \cdot F_{in} + F_{in} \cdot F_{out}$, typically yielding significant savings for non-trivial kernels and moderate channel sizes.
Why this matters for wearables: Lower MACs reduce inference latency and CPU active time, which directly impacts energy drain. Smaller models reduce flash footprint and leave room for sensing, buffering, and application logic, enabling always-on operation.

4 System Level

We design an end-to-end pipeline for classifying stress-related patterns from raw PPG signals on an embedded microcontroller. In this work, PPG samples are streamed to the MCU over a serial link, then processed in firmware using a lightweight high-pass filter to remove baseline drift. The filtered stream is stored in a ring buffer, and a sliding-window segmentation scheme (window length N with a configurable hop size) is used to form inference windows. For each window, the firmware applies per-window normalization (mean and standard deviation) before feeding the data to a TinyML model deployed with TensorFlow Lite Micro. The TinyML-based network extracts discriminative temporal features and outputs a stress versus non-stress decision, which is reported via serial logs and displayed on the monitor or device.

5 Hardest Problems and Responses

5.1 Self-collected Data (Week 3's Task)

Issue. A key challenge is that our dataset is self-collected using only a single MAX30105 sensor, without auxiliary devices (e.g., ECG, EDA, or respiration sensors) to provide complementary physiological signals. Moreover, obtaining reliable ground-truth stress labels is difficult because emotional states are inherently subjective and hard to control during data collection. Repeated exposure to the same stimulus can also attenuate the intended emotional response, leading to label noise. For example, watching the same horror video multiple times may no longer induce anxiety, even though the stimulus category remains unchanged.
Response. We collected data over an extended period to better capture stable and representative emotional states. Initially, we used horror films as stimuli to induce stress, but repeated exposure reduced their effectiveness and made the resulting labels less reliable. We therefore additionally recorded PPG signals during an exam, which provided a more natural and consistent stress condition. Overall, we were only able to obtain approximately 800 reliable samples.

5.2 Model's Resource Consumption Measurement (Week 5's task)

Issue. We aim to measure model resource consumption comprehensively. While Flash and RAM usage are readily available, and inference latency can be measured directly in Arduino code, energy and power

consumption are not exposed by default. Accurately profiling these metrics typically requires external measurement equipment (e.g., a power monitor), which adds additional hardware and setup complexity.
Response. We therefore use proxy metrics to indirectly estimate energy and power consumption, as described in Sec. 2.4. However, it is essential to clearly understand what these proxies actually measure. After extensive verification and repeated code refinements to ensure correctness, we use these metrics as practical surrogates for tracking energy and power.

5.3 Buffering

Issue: Buffer management is another key challenge. We must implement reliable write and read operations for streaming sensor data to prevent sample loss. In addition, selecting an appropriate buffer size is non-trivial, as it requires balancing data reliability against limited memory resources and overall utilization on the MCU.
Response: We use a robust producer-consumer design: it drains the MAX30105 FIFO into a ring buffer, then emits samples at a fixed Fs Hz schedule. Overflow is handled deterministically via drop-oldest to preserve real-time freshness, and underflow is handled via repeat-last to maintain a stable output rate. A bounded catch-up loop mitigates timing jitter without risking starvation, and finger-state gating resets the window buffer to prevent invalid segments from contaminating inference windows. Together, these mechanisms make the streaming and windowing behavior reliable under typical MCU timing variability.

6 Future Plan

Because my background is not in real-world implementation such as hardware or PCB design, system-level deployment, or building real-time products, I am motivated to strengthen these fundamentals to better support my PhD trajectory. I therefore plan to start preparing well before Fall 2026, focusing on building the practical foundation needed to catch up and contribute effectively. Building on the next project, I will learn by doing and continuously improve my skills. This is my proposed plan for the next 6-12 months:
Before the Fall 2026 Semester

- **March–May (2026):** Join Dr. Vu's project as a part-time research assistant to strengthen my background knowledge and hands-on skills. During this period, I will commit **4–6 hours per day** while completing remaining responsibilities in my current lab. I will provide regular progress reports, and my work will be monitored and evaluated under Dr. Vu's supervision.
- **June–August (2026):** Continue the ongoing project or transition to a new project aligned with potential PhD research directions, depending on Dr. Vu's plan. During this period, I am willing to work as a full-time research assistant with a commitment of **8–10 hours per day**.

After the Fall 2026 Semester

- Begin PhD study and conduct research under Dr. Vu's guidance.

7 Research Idea from Week 6 and 7

During Weeks 6 and 7, I think about exploring an end-to-end design for an efficient healthcare monitoring wearable. The core idea is to make energy and power consumption context-aware and selective. Under normal conditions, when physiological signals and observations change only slightly, the device should remain in a low-power mode and perform minimal computation. When significant fluctuations or abnormal

patterns are detected, the device dynamically increases its compute budget and sampling intensity to enable more accurate tracking and timely decisions. This event-driven behavior is conceptually aligned with neuromorphic and adaptive sensing principles. Implementing such a system would require co-optimizing both software and hardware, and could represent a meaningful step toward practical, always-on wearable healthcare monitoring with substantially improved energy efficiency.